ADVENTIST UNIVERSITY OF CENTRAL AFRICA

BIG DATA ANALYTICS · FINAL PROJECT · NOVEMBER 2026

Distributed Multi-Model Analytics for E-Commerce Data

Designing MongoDB, HBase and Apache Spark around the questions each is built to answer — then combining all three in one analysis.

AUTHOR
Thierry SEBAKUNZI

STUDENT ID
101394

SOURCE CODE
github.com/code-gnosis/auca-ecommerce-multimodel-analytics

2,000,000	20.0M	500,000	3.9 GB	853 MB
SESSIONS	PAGE VIEWS	TRANSACTIONS	RAW JSON	PARQUET

1 System architecture

Three stores, each chosen for a shape of question rather than a preference.

The dataset is 3.9 GB of JSON describing 90 days of trading: 10,000 users, 5,000 products across 25 categories, 2 million browsing sessions containing 20 million page views, and 500,000 transactions made of 954,134 line items. No single engine answers every question asked of that data well, which is the premise of the project.



THE SELECTION RULE

MongoDB holds what is read as a whole object: a product, a customer, a transaction with its line items, a session with its journey. **HBase** holds what is read as a range over time for one key: this user's recent sessions, this product's daily performance. **Spark** does everything that touches all two million sessions at once, and every join across the two stores — because neither database can join to the other, and HBase cannot aggregate at all.

2 Data modelling

2.1 MongoDB — document model

Four collections. The decisions worth defending are the three places where the schema departs from the raw files.

COLLECTION	DOCUMENTS	DESIGN DECISION
`products`	5,000	**Category name copied onto every product.** Denormalised deliberately: every product query wants the category label, and one redundant string per product is cheaper than a `$lookup` on every read. Categories change rarely, so the duplication is safe. `current_price` is also precomputed from the last entry of `price_history`, so no reader sorts that array.
`users`	10,000	**Purchase summary embedded.** Order count, lifetime value, average order value and first/last order dates are computed once at load. "Show me this customer" becomes a single document read with no joins — the query a dashboard makes constantly.
`transactions`	500,000	**Line items embedded.** A transaction is always read as a whole and its items have no independent life, which is the textbook case for embedding rather than referencing.
`sessions`	200,000	**Whole journey in one document.** Entry point, every page view, cart contents and outcome, with no joins. A subset, because Spark reads the full two million and MongoDB only needs enough to answer journey questions.

Indexes follow the queries, not the fields: `items.product_id` on transactions (a multikey index into the embedded array), `user_id + timestamp` descending for customer history, and `viewed_products` on sessions.

2.2 HBase — wide-column model

Two tables. In HBase the row key *is* the design, because it fixes the physical sort order and therefore which questions are cheap.

TABLE	ROW KEY	WHY THIS KEY
`user_sessions`	`user_id # reverse_timestamp`	HBase stores rows in lexicographic order and a scan walks forward. Reversing the timestamp (`9999999999999 − epoch_ms`, zero padded) puts the **newest session first**, so "the last 10 sessions for this user" stops after 10 rows rather than reading a whole history and sorting it. Putting `user_id` first keeps one user's rows contiguous in one region.
`product_daily`	`product_id # yyyy-mm-dd`	Date **forward** here, not reversed, because the question is a date range: "how did this product do across March" is a scan between two row keys.

Column families are split by read pattern, not by topic.
`user_sessions` has `s` (summary: times, device, geography, referrer, outcome — small, always read) and `p` (page view counts, products viewed, cart — larger, read only on drill-in). HBase stores each family in its own files, so the common query reads only `s`. Query 2 in `query_hbase.py` demonstrates exactly this.

`product_daily` uses one family `m` with one sparse cell per metric. A product with no sales on a given day simply has no purchase cells, and HBase stores nothing for them — which is the reason to use a wide-column store rather than a table with five nullable numeric columns.

> **WHY THIS DATA WENT WHERE**
>
> Transactions live in MongoDB because they are read whole and have rich nested structure. Session *history per user* lives in HBase because the access pattern is a time-ordered range for one key, which a document store answers only by index lookup and sort. The same session data exists in both: MongoDB holds a subset for journey analysis, HBase holds the time series. That is duplication by design, not by accident — each copy is shaped for its own question.

3 Spark processing pipeline

3.1 The format problem, and the fix

The generator writes each file as one JSON array. Spark can read that only with `multiLine=true`, and a multiLine file **cannot be split**: one 188 MB file becomes one task that must hold the entire parsed array. With twenty such files the job first crawled and then failed with `SparkOutOfMemoryError`.

`to_jsonl.py` converts the arrays to JSON Lines in a single streaming pass taking 45 seconds. JSON Lines is splittable, because a line boundary is a safe place to cut, so one file is shared across tasks. That conversion is paid once and saves time in every job afterwards.

> SCHEMA INFERENCE TRAP — FOUND AND FIXED
>
> `cart_contents` is a JSON object *keyed by product id*. Spark infers schema from data, so a different product id in each document became a different **field**: a struct with 5,000 named columns, which broke every query that touched the cart. Dynamic keys are data, not schema. The conversion step rewrites the cart as an array of `{product_id, quantity, price}`, giving Spark one stable type.

3.2 Cleaning and normalisation

`batch_clean.py` reads the JSON Lines once, applies the rules below, and writes Parquet — columnar, compressed and splittable. 3.9 GB of JSON becomes 853 MB, a 4.6× reduction before any query runs.

- **Timestamps** parsed from ISO strings into real timestamp columns.
- **Duplicates** dropped on natural keys. Four duplicate session ids were found in two million rows — a real defect, not a rounding artefact.
- **Missing values kept and flagged, not dropped.** 243,389 transactions (48.7%) have no `session_id`: a purchase with no recorded browsing. Dropping them would remove half the revenue; they are flagged with `has_session` instead.
- **A consistency check** that `subtotal − discount = total`. Zero rows failed, which is worth stating: the check ran and passed.
- **Derived columns** computed once — hour of day, day of week, cohort month, device fields lifted out of the nested struct — so no later job recomputes them.

3.3 Analytical jobs

Product affinity self-joins the line items on transaction id, keeping `product_a < product_b` so each pair appears once, which halves the shuffle. It reports support, confidence and lift. **Cohort analysis** groups customers by registration month and tracks spending by `months_since`, so every cohort's month zero aligns in the same column regardless of calendar date.

Optimisation choices made and worth naming: `spark.sql.shuffle.partitions` set to 8 rather than the default 200, because 200 tiny tasks cost more to schedule than to run; `cache()` on the line items and transactions, which both jobs read repeatedly; and the Parquet conversion itself, which is the single largest saving.

A PARTITIONING DECISION THAT WAS REVERSED

Writing sessions partitioned by date looks obviously right. It failed twice. Without a repartition, each of 20 tasks held an open writer for all 90 dates — 1,800 buffers, and an out-of-memory error. With a repartition, the fix cost a full shuffle of two million documents whose page-view arrays had to be serialised and moved, which took longer than every query the partitioning would have accelerated. At 853 MB a full scan is already cheap and column pruning does most of the work. On a cluster holding hundreds of gigabytes the trade reverses and partitioning wins.

4 Integrated analytics: customer lifetime value

One question that no single store can answer.

Business question. Which customers are worth retaining, and which are only browsing? Marketing needs a ranked list with a reason attached, not a spend total.

STEP	SYSTEM	WHAT IT CONTRIBUTES
1	MongoDB	Profile and spend. One aggregation pipeline over `users`, reading the embedded purchase summary rather than recomputing from transactions.
2	HBase	Engagement. A scan of `user_sessions` reading only the `s` family — session count, duration, recency. Rows arrive newest-first per user, so the most recent session is the first row seen.
3	Python / Spark	The join and the score. Neither database can reach the other.

```
CLV = avg_order_value  x  purchase_frequency  x  expected_lifetime_months

  avg_order_value      MongoDB, embedded summary
  purchase_frequency   orders per active month
  expected_lifetime    6 months x engagement factor, capped at 24
                       engagement factor from HBase session count and conversions
```

The run joined **10,000 customers from MongoDB** with engagement for **9,998 of them from HBase**. It separates 9,712 engaged customers from 286 who have spent well but gone quiet — the actionable split, and one that neither spend data nor engagement data alone would produce.

STATED HONESTLY

Total predicted CLV is 2.55 billion against 466 million actually spent — 5.5×. That is the model multiplying by an assumed 20-month lifetime while the data covers 90 days. The *ranking* is the usable output; the absolute figure is not. The weights are a judgement, and every raw input is reported beside each score so the effect of the weighting stays visible.

5 Findings

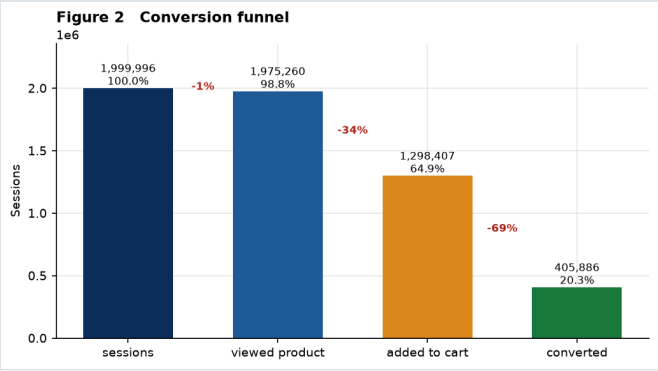


Figure 1. 64.9% of sessions put something in a cart; 20.3% buy. The 69% loss at that single step is larger than every other stage combined.

The cart is where the money leaks. Getting people to view products is effectively solved at 98.8%, and adding to cart holds 64.9%. Then two thirds of those carts are abandoned. Any optimisation effort belongs at checkout, not at the top of the funnel.

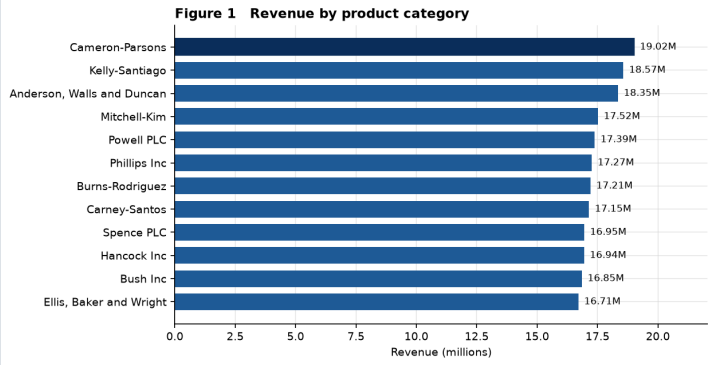


Figure 2. Revenue of 409.5 million spread across 25 categories, with the largest taking only 4.64%.

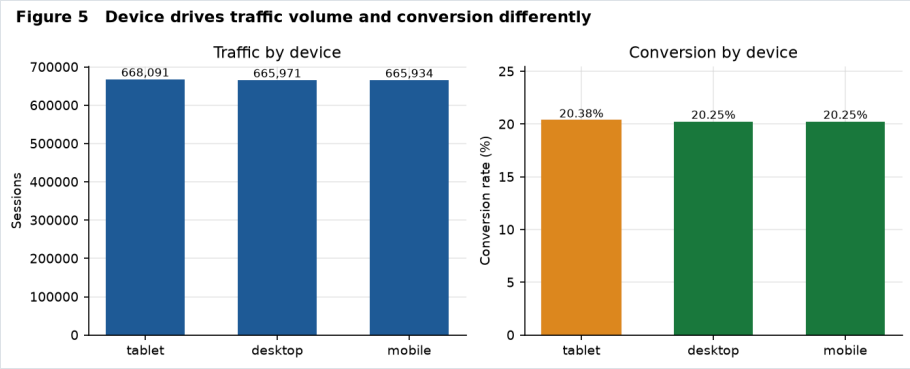


Figure 3. Mobile 20.25%, desktop 20.25%, tablet 20.38%. Device makes no measurable difference here.

That flatness is itself the finding. A real platform shows a pronounced mobile conversion penalty; this dataset does not, because device is assigned at random by the generator. The analysis is correct and the data has no signal to find — a distinction worth keeping clear.

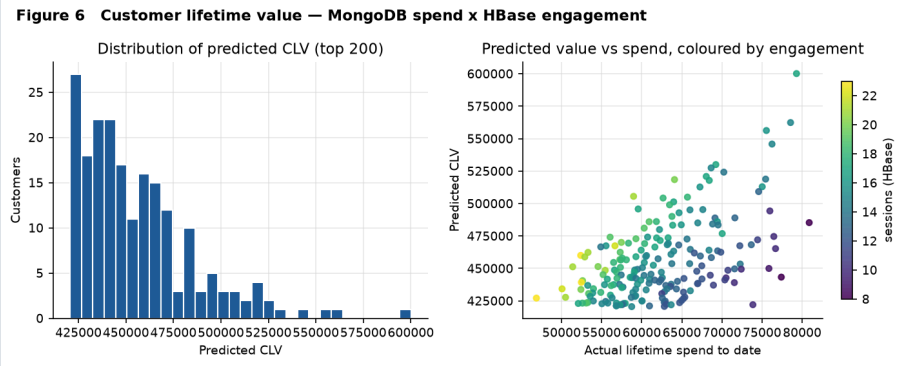


Figure 4. Predicted CLV from MongoDB spend crossed with HBase engagement. Colour is session count: at equal spend, more engaged customers score higher.

VALUE TIER (LIFETIME SPEND)	CUSTOMERS	SHARE	AVG ORDERS	AVG ORDER VALUE
Under 30,000	134	1.34%	35.8	776.88
30,000 – 40,000	1,935	19.35%	42.7	861.06
40,000 – 50,000	4,635	46.35%	49.0	925.77
50,000 – 60,000	2,782	27.82%	55.2	983.39
Over 60,000	514	5.14%	61.9	1,038.85

The two effects compound. Customers in the top tier do not merely buy more often — their average order is 34% larger than the bottom tier's. Frequency and basket size move together, so the top tier is worth more than order count alone suggests.

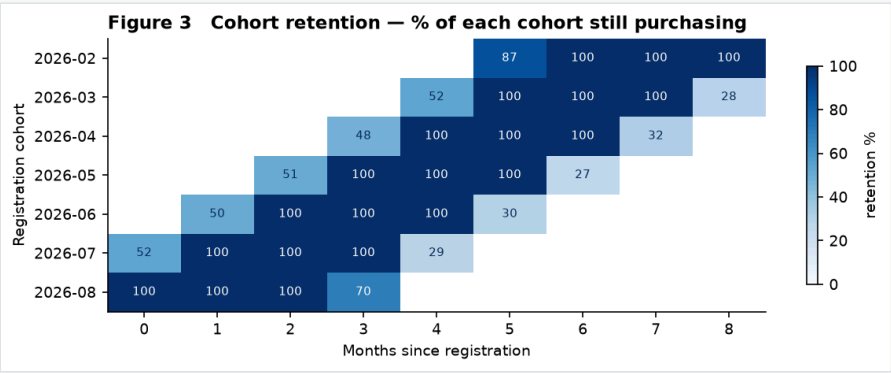


Figure 5. Retention by registration cohort. Beyond the partial first month it sits at essentially 100%, because the generator gives every user continuous activity. No cohort churns.

Product affinity could not be measured reliably, and saying so is the result. 14,679 product pairs clear a minimum co-occurrence of two, but the highest co-occurrence anywhere in 439,067 baskets is five. With 5,000 products, 12.5 million possible pairs and an average basket of 1.9 items, lift values above 1,000 are arithmetic artefacts of sparsity. Both rankings are reported — by lift, and by co-occurrence count — with the volume ranking marked as the only defensible one.

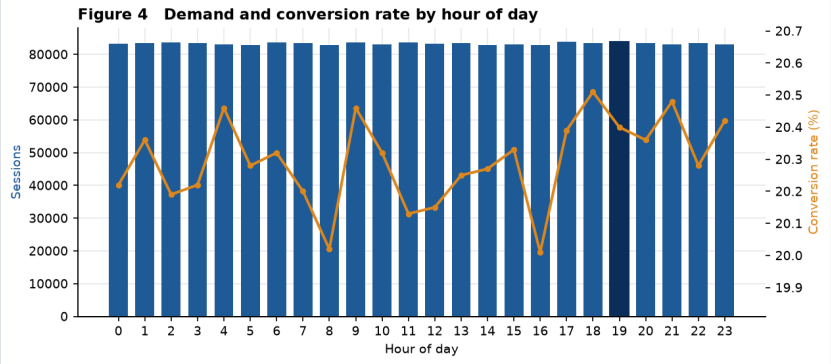


Figure 6. Demand peaks at 19:00 (83,936 sessions) and conversion at 18:00 (20.51%). The range across all 24 hours is under 1.5%.

6 Scalability

COMPONENT	CURRENT	WHAT CHANGES AT 100×
Spark	`local[4]`, 10 GB driver	The code is unchanged on a cluster: `local[4]` becomes a YARN or Kubernetes master. Date partitioning, removed here as a net loss, becomes the right choice once a full scan is no longer cheap.
HBase	Standalone, one node	The row keys already distribute: `user_id` first spreads load across regions. The risk is hot-spotting if keys were sequential — they are not. Salting the prefix would be the next step if one user ever dominated.
MongoDB	Single instance	Shard on `user_id` for users and sessions, on `timestamp` for transactions. The embedded summaries keep working, since nothing depends on a cross-shard join.
Storage	3.9 GB JSON, 853 MB Parquet	Parquet with column pruning is already the lever that matters; the next one is partitioning by date, then by category.

7 Limitations and future work

- **The data is synthetic and uniformly random.** Cohort retention is 100%, every customer is a frequent buyer, no product outsells another and device has no effect. Several analyses here demonstrate method rather than discover behaviour, and the report says so where that applies rather than presenting noise as insight.
- **CLV is a ranking, not a forecast.** 5.5× actual spend, for the reason given in section 4.
- **HBase runs standalone**, so region splitting, compaction and real distribution are not demonstrated — only the schema and access patterns are.
- **Spark runs on one laptop.** The code is cluster-ready; no cluster behaviour is shown.
- **Next.** Real Spark connectors to MongoDB and HBase instead of a Python join; a sessionised funnel that follows individual users across sessions rather than within one; and FP-Growth for affinity, which handles sparsity better than pairwise lift.

8 Methodology note

Every figure in this report was produced by a script in the repository and written to `results/` as JSON before being charted. No number was transcribed by hand, so the tables, the figures and the code cannot disagree. The full pipeline is reproducible from a clean machine with the commands in the repository README.

Thierry SEBAKUNZI · Student ID 101394 · Adventist University of Central Africa · Big Data Analytics Final Project · November 2026
Source code and reproduction instructions: github.com/code-gnosis/auca-ecommerce-multimodel-analytics